Vaulta

SECURITY ASSURANCE REPORT

What Vaulta guarantees, and the evidence behind it

Architecture, cryptography, live verification results, findings and open items for the Vaulta macOS client and its control plane.

VERSION	DATE	CLASSIFICATION	AUDIENCE
1.0	14 November 2026	Public	Customer security review

DOCUMENT CONTROL

Document	Vaulta Security Assurance Report
Version	1.0, issued 14 November 2026
Scope	Vaulta macOS client (control process and sandboxed session process) and the Vaulta control plane, as built on the date above
Basis	Source review, the automated test suites, and live verification runs carried out on 14 November 2026
Assessment type	First-party assessment. This is not a certification or an independent third-party audit.
Classification	Public. Prepared for the security, risk and legal teams of current and prospective customers.
Companion documents	Threat model, software bill of materials, architecture decision records, deployment guide, public security brief

CONTENTS

Executive summary 3

1 Scope and method 4

2 Architecture and trust boundaries 6

3 Data handling 7

4 Session lifecycle and shred 8

5 Network egress 10

6 Vault cryptography 11

7 The attestation receipt 12

8 Supply chain and models 13

9 Control plane 14

10 Device identity and updates 15

11 Findings log 16

12 Known limits and open items 18

A Evidence log 19

B Verifying our claims yourself 21

C Glossary 22

EXECUTIVE SUMMARY

Executive summary

Vaulta turns confidential calls into reviewed memos on the Mac itself. The audio and the verbatim transcript stay in an encrypted, memory-pinned vault, inside a process that has no network access and is destroyed when the memo is approved. Every session ends with a receipt signed by the Mac's Secure Enclave.

VERIFIED LIVE
27
Observed working on a real run, each with a control proving the check could fail.

OPEN ITEMS
15
Not done or not yet verified (section 12). None is an unresolved cryptographic defect.

DESIGN LIMITS
7
Deliberate boundaries of what the product protects against.

The five statements a reviewer can test

STATEMENT	HOW IT IS ENFORCED	STATUS
One process touches the meeting, and it has no network.	The session process runs in the macOS App Sandbox with no network entitlement. The OS refuses its sockets.	VERIFIED
Verbatim data lives in an encrypted, pinned vault.	Per-record AES-256-GCM in `mlock`ed memory, zeroed and read back at approval, timeout, sleep, signals or loss of the control link.	VERIFIED
The process then exits.	The kernel zeroes an exited process's pages before reuse, which removes copies the vault cannot reach.	VERIFIED
Every session ends with a signed receipt.	A non-exportable Secure Enclave P-256 key signs it. The control plane refuses software-signed receipts by default.	VERIFIED
We list what is not done.	Signing and notarization, live meeting capture, a live identity-provider tenant and other gaps are in section 12.	POLICY

What this assessment found and fixed

Live testing on 14 November 2026 surfaced six issues: three fixed in code and re-verified the same day, one mitigated through deployment guidance, and two open (section 11).

ID	FINDING	SEVERITY	STATUS
F-01	A refused vault write could hand its AES-GCM nonce to the next record	MEDIUM	FIXED
F-02	An unusable device or memo key was silently replaced with a new one	MEDIUM	FIXED
F-03	A revoked Mac could never be enrolled again in the same organization	LOW	FIXED
F-04	Under the web server's defaults, forwarded headers from the local host were trusted	LOW	MITIGATED
F-05	Two recorded model hash pins could not be reproduced	LOW	OPEN
F-06	The public brief overstated protection against debugger attach	INFO	OPEN

SECTION 1

1 Scope and method

We assessed the product as built, not the roadmap. Each claim names the evidence behind it, and each live check included a control that shows the check can detect a failure.

> **How to read the evidence**
>
> Live checks ran on one Apple-silicon Mac (macOS 27.2) with demo audio; some vault checks ran the real vault code in a test program, and each such case is labelled. Verified means observed on a real run, not proven for every Mac or every call.

In scope

MAC CLIENT

- Control process (menu-bar app)
- Session process (capture, speech, memo model, review window)
- The volatile vault and shred
- Receipt signing, device keys, approved-memo store
- Model download and verification

CONTROL PLANE

- Enrollment, devices, revocation and Deal Room workspaces
- SCIM 2.0 deprovisioning
- Admin console, admin audit log and its hash chain
- Receipt intake and audit exports (JSON, CSV, signed PDF)
- Production hardening on PostgreSQL 16

Test environment

Hardware and OS	One Apple-silicon Mac, macOS 27.2, Secure Enclave available
Client builds	Debug, release and ThreadSanitizer builds; the shipping app built with the production sandbox entitlements and ad-hoc signed
Control plane	FastAPI service on SQLite and on a throwaway PostgreSQL 16 cluster, reached over real HTTP
Audio	The bundled demo call. Live Zoom or Teams capture is not part of this assessment.
Automated suites	Client: 334 tests pass (1 skipped). Control plane: 248 pass on PostgreSQL 16; 246 pass on SQLite (2 PostgreSQL-only tests skipped).

Evidence levels

LEVEL	MEANING
VERIFIED	Observed working on a real run of the product or of its real components, with a control that proves the check can fail.
OPEN	Not done, or not yet verified. Listed with what is missing.
DESIGN	A deliberate limit of what the product protects against.

Principles we held to

A CHECK MUST BE ABLE TO FAIL

Every live check was paired with a positive control. For example, plaintext was placed in a pinned page on purpose to prove the memory scan would report it, and an unlocked counter was raced on purpose to prove the duplicate check and ThreadSanitizer would catch it.

LIMITS ARE PART OF THE RESULT

Where a run covered less than the claim, such as a debug build only, a test program, or a simulated identity provider, the limit is stated next to the result rather than in a footnote.

INDEPENDENT WITNESSES

Where possible a second, independent mechanism confirmed a result: Python's AES-GCM against the vault's ciphertext, the macOS `nettop` monitor alongside the socket watchdog, and ThreadSanitizer alongside the stress tests.

NOTHING LEFT BEHIND

Temporary test drivers, keychains, certificates, databases and exported keys were removed after each run, and the source tree was confirmed clean.

Severity scale used in section 11

SEVERITY	DEFINITION
MEDIUM	Weakens a stated security property in a reachable code path, without a demonstrated way to exploit it within the threat model.
LOW	Needs local access or unusual conditions, affects availability or record-keeping, or is reduced by configuration.
INFO	A documentation or communication accuracy issue with no effect on the product's behaviour.

SECTION 2

2 Architecture and trust boundaries

The client is split so that the only code that ever sees verbatim audio or text cannot reach the network, and is destroyed when its job is done.

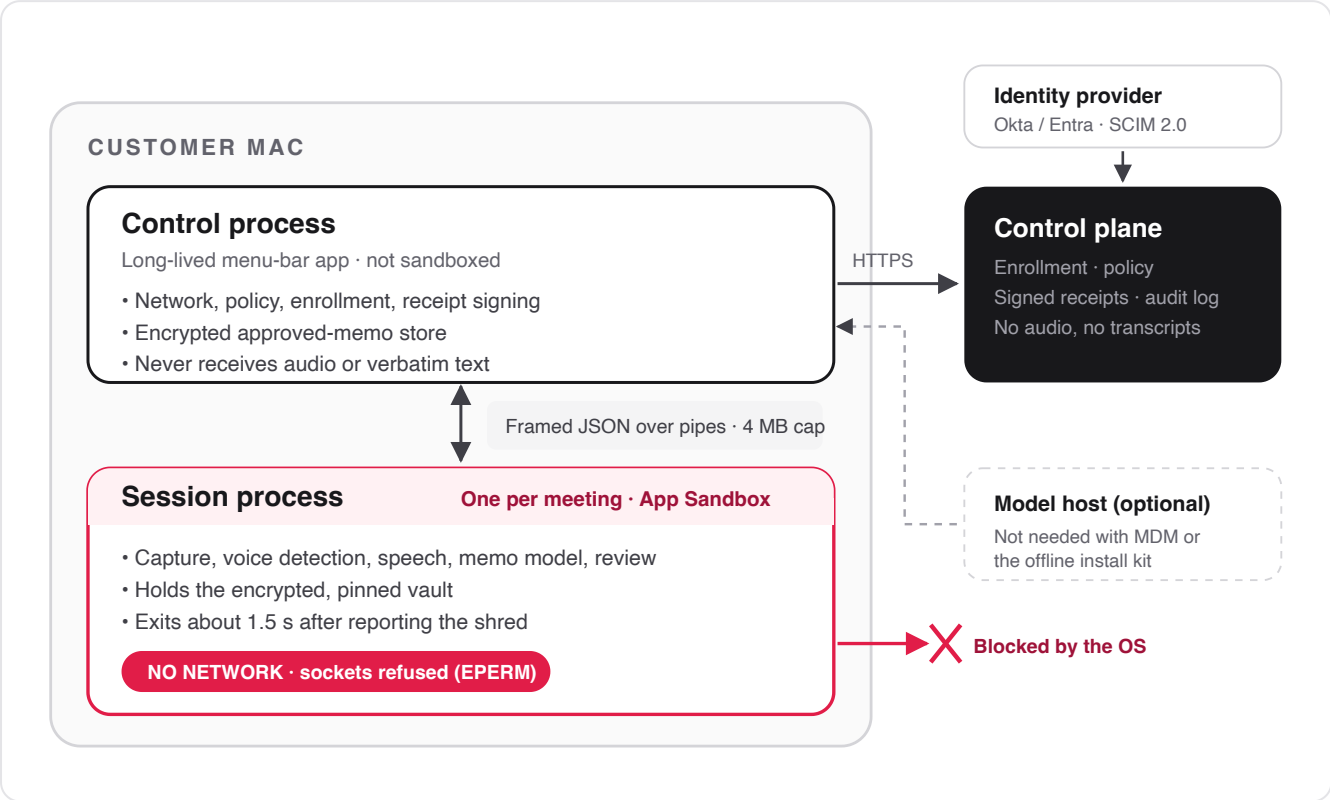


Figure 1. Two processes, one boundary. Verbatim data never leaves the sandboxed session process; the control process handles everything that needs the network.

Trust boundaries

BOUNDARY	WHAT CROSSES IT	WHAT DOES NOT
Session → control	The approved, sanitized memo, a shred report and model hashes, as versioned, size-capped, sequence-checked frames	Audio, verbatim transcript, the vault key
Control → control plane	Enrollment, signed receipts (metadata only), policy and glossary sync over HTTPS	Audio, transcripts, memo content
Identity provider → control plane	User status over SCIM: user name, emails, display name, external ID, active flag	Anything from the Mac
Session → internet	Nothing. The sandbox has no network entitlement and refuses every socket connection.	

Why the session is a separate, killed process

The vault can zero its own bytes, but not every copy the process makes along the way: audio buffers, decrypted reads, model tensors and strings. When a process exits, the kernel zeroes its pages before reuse, which removes those copies too, including ones that cannot be enumerated. That is why the design kills the process rather than promising to scrub every buffer (architecture decision record 0001).

SECTION 3

3 Data handling

Every category of data the product touches, where it lives, and whether it leaves the Mac.

DATA	WHERE IT LIVES	LEAVES THE MAC?
Raw audio	Session process memory, sealed into the vault within milliseconds	**No.** The session process cannot open sockets.
Verbatim transcript	The vault, and the Review window inside the session process	**No.**
Approved memo	Control process. One AES-256-GCM file per memo with its own key, wrapped to a Secure Enclave key.	Only if the user exports it (Markdown or Word)
Signed receipt	Written to disk first, then uploaded. Offline receipts wait in an outbox.	**Yes, metadata only:** session ID, device hardware UUID, workspace code name, timestamps, model hashes, verdicts
Enrollment and policy	The control plane	Yes. Account and policy metadata.
Directory status	The control plane, pushed by the identity provider over SCIM if connected	Not from the Mac
Admin audit log	The control plane: who changed what, when, and from which address. Never tokens or keys.	Not from the Mac
Logs	Local. Counts and timings only, never transcript text.	**No.**
Keys	Secure Enclave blobs, useless on another Mac	**No.**

> **Classify these as identifiers**
>
> Receipts carry the device hardware UUID and a workspace code name. Treat both as identifiers when classifying the control plane's data.

What is written to disk

WHAT	WHERE
Application	`/Applications/Vaulta.app`, with the session process as a helper inside it
Models (read-only)	`/Library/Application Support/Vaulta/Models`
Approved memos	`~/Library/Application Support/Vaulta/Memos`, encrypted, one file each
Receipts and outbox	`~/Library/Application Support/Vaulta/Receipts` and `Outbox`
Logs	`~/Library/Application Support/Vaulta/Logs`, counts and timings only
Keys	`~/Library/Application Support/Vaulta/Keys`, Secure Enclave blobs

Deleting the keys makes every stored memo permanently unreadable. That is by design and is the offboarding lever.

SECTION 4

4 Session lifecycle and shred

A session has one ending: the vault is zeroed and verified, the session process exits, and the control process signs a receipt that says exactly what happened.

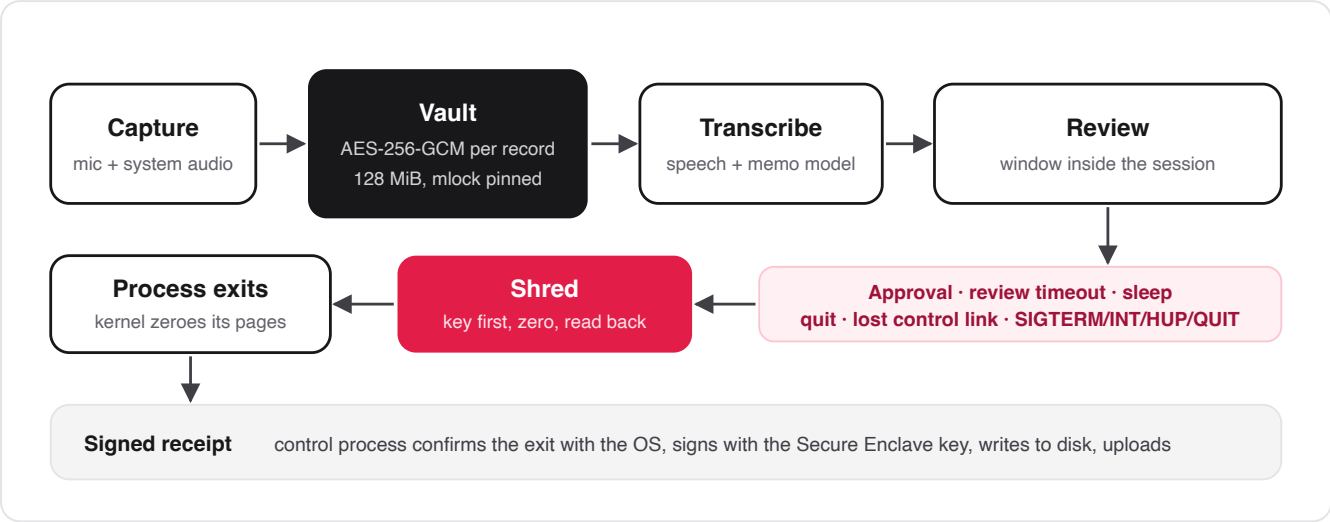


Figure 2. Every path out of a session ends in a verified shred, a confirmed process exit and a signed receipt. SIGKILL cannot be caught; the kernel still zeroes the pages and the receipt records the shred as not verified.

What the vault guarantees

PROPERTY	MECHANISM AND EVIDENCE	STATUS
Not written to swap or a core dump	`mlock` plus a zero core-size limit; fails closed if either fails. A real sandboxed session reported its 128 MiB vault as kernel-pinned.	VERIFIED
No plaintext in pinned pages	A memory scan using the kernel's pinned flag found no plaintext in any pinned region, in debug and release builds.	VERIFIED
The key lives only in a pinned page	The real key was found in exactly one pinned page, before and after 300 decryptions, and nowhere after the shred (debug build).	VERIFIED
Shred zeroes key and data, then verifies	Key first, then `memset_s`, then a read-back. Every real run read the vault back as zero.	VERIFIED
Timeout, sleep and abort all shred	Review timeout and a forced sleep were run with the built app. Only an explicit approval records `human_attorney_reviewed`.	VERIFIED
Termination signals shred first	SIGTERM, SIGINT, SIGHUP and SIGQUIT zero and verify the vault, then exit. Run live during capture and review.	VERIFIED
Killed sessions get an honest receipt	Killing the session gave a receipt that is never clean. If the control process is killed, the session shreds and the next launch issues an aborted receipt.	VERIFIED

Copies outside the vault

These exist and are handled by process exit, not by the vault.

Audio in flight	CoreAudio buffers, resampler output and a small frame accumulator, for milliseconds before sealing

Decrypted reads	CryptoKit decrypts into ordinary memory, so each decrypted record exists there until reused
Model working memory	Speech-model tensors, the memo model's prompt and cache, and strings made from transcript text
System audio services	`coreaudiod` and the audio HAL hold buffers outside our process and our control
Swap and hibernation	Unpinned copies can be paged out; swap is encrypted by default. A sleep hook shreds; lid-close and a full hibernation image are untested.

The claim we make, and the one we do not

We claim that raw audio and verbatim transcripts are held only in encrypted, pinned buffers, which are zeroed and verified at approval, timeout or sleep, and that the process that handled them then exits. We do not claim that no trace of the conversation remains anywhere, and our receipts do not claim it either.

SECTION 5

5 Network egress

Three independent layers keep the session process off the network. The operating system enforces the first; the other two are witnesses.

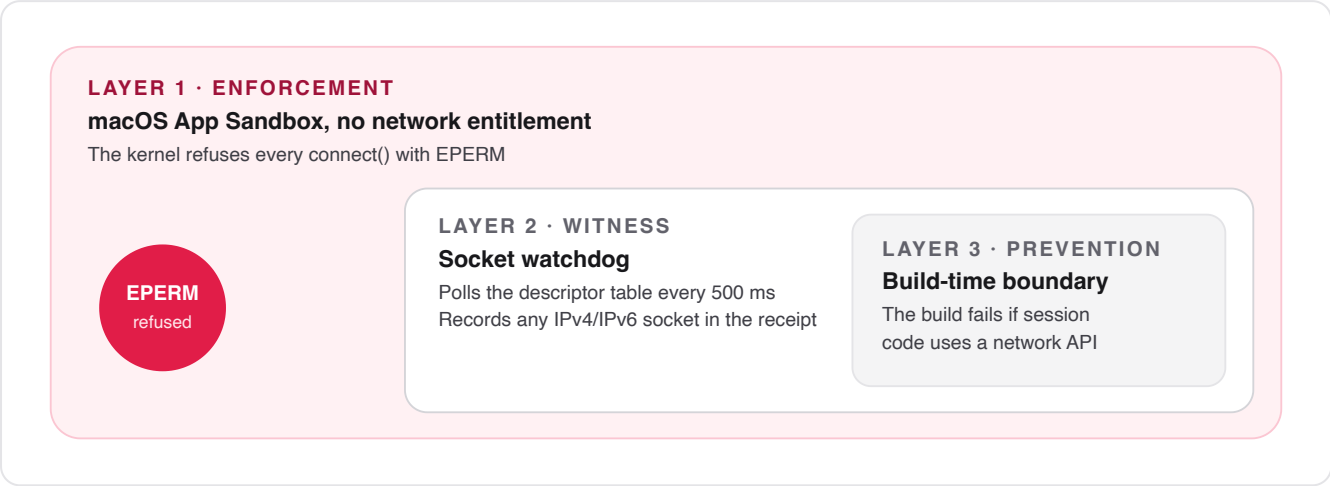


Figure 3. Defence in depth. The sandbox is the control; the watchdog and the build boundary are independent witnesses.

Evidence

CHECK	RESULT	STATUS
Sandbox enforcement	A sandboxed session launched by the control process got `EPERM` on connect; the same code unsandboxed reached the internet. A deliberate connect from a binary with the production entitlements also got `EPERM`.	VERIFIED
Full sessions, zero sockets	Seven full runs on one Mac, all on the demo call, reported zero internet sockets and a shred verified as zero.	VERIFIED
Independent traffic monitor	The macOS `nettop` tool recorded no network flows for the session process in five consecutive runs. It was first shown to record a renamed process that moved 8 MB.	VERIFIED
Watchdog detection	Inside the production sandbox it ignored a local-IPC socket and flagged a TCP socket even though the sandbox refused its connection.	VERIFIED
Build-time boundary	A `URLSession` call planted in session-side code failed the build test, which named the file.	VERIFIED

Dependencies with download code

WhisperKit and swift-transformers link Hugging Face download code that cannot be trimmed from the dependency graph. Model loading is told not to download and fails if a file is missing, the sandbox refuses any unintended call, and the full-run results above show no socket was ever opened.

Limits

The watchdog samples, so it sees sockets, not traffic; a socket opened and closed between two samples is stopped by the sandbox but not seen. All runs used demo audio on one Mac. The receipt never reports a byte count, because macOS exposes none per process without private API.

SECTION 6

6 Vault cryptography

Every record is sealed with AES-256-GCM under a key that exists only in a pinned page. The nonce and the authenticated data make every record unique and tie it to its place in the vault.

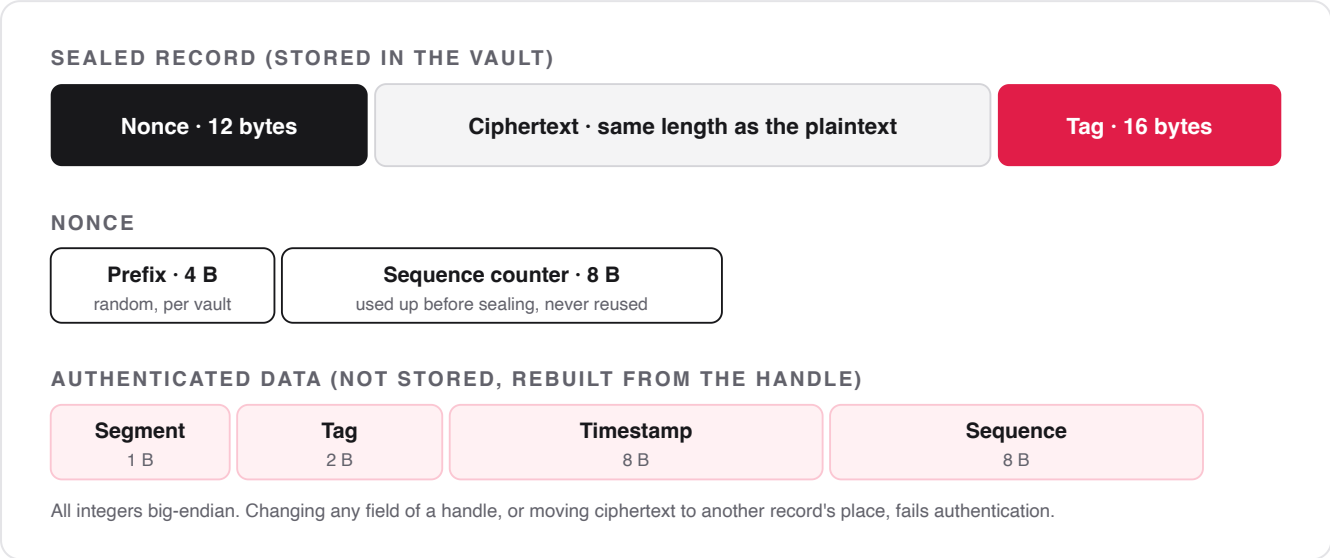


Figure 4. The sealed record format. The nonce makes every encryption unique; the authenticated data binds each record to its segment, tag, timestamp and sequence.

Evidence

PROPERTY	WHAT WAS RUN AND WHAT HAPPENED	STATUS
Ciphertext cannot be swapped or re-pointed	All 47 forged handles and all 130 single-byte changes to sealed records in memory were refused. An independent Python AES-GCM implementation confirmed that each of the four fields is authenticated, and refused all 20 pairings of one record's ciphertext with another record's data.	VERIFIED
No nonce reuse	1,131,072 stored records had 1,131,072 distinct nonces. With 12 concurrent writers and a filling segment, over a million attempts, most of them refused, produced no repeated sequence, and the next sequence always equalled the number of attempts. See finding F-01.	VERIFIED
Fresh key per vault	3,000 vaults produced 3,000 distinct keys and prefixes; the share of 1-bits was 0.500.	VERIFIED
Safe under concurrent writers	12 writers and 4 readers made over 270,000 appends with no repeated or missing sequence and no corrupted record. 450 shred-during-write trials left no late writes and no leftover bytes. ThreadSanitizer, with the vault's Swift and C code instrumented, reported no data race.	VERIFIED

Limits

These checks ran the real vault code inside test programs, in debug, release and ThreadSanitizer builds, not inside the shipping session process. Reading the key or writing into vault memory requires a debug build. The attacker these checks model is one that can forge handles or write to the vault's memory, not a remote one.

SECTION 7

7 The attestation receipt

After every session the control process builds a receipt, signs it with a non-exportable Secure Enclave key, and uploads it. The control plane verifies it before counting the session.

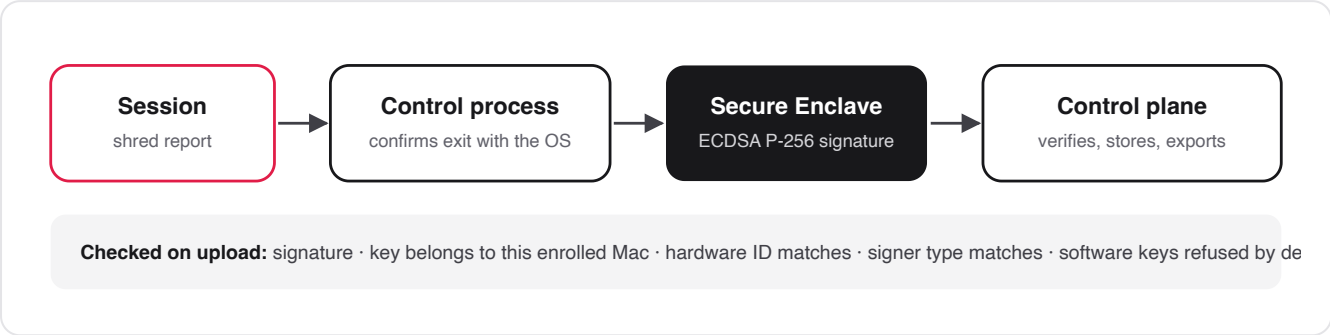


Figure 5. The receipt is written to disk before upload; an offline Mac keeps receipts in an outbox until it reconnects.

Fields

`session_id`	One per meeting
`device_hardware_uuid`	The enrolled Mac
`workspace_code_name`	Deal Room code name
`capture_started_at`, `ram_shredded_at`	UTC, to the second
`asr_model_hash`, `slm_model_hash`	Model weights that ran
`internet_sockets_observed`, `egress_verdict`	From the watchdog
`memset_zero_verified`	Vault read back as zero
`session_process_exited`	Confirmed by the OS
`human_attorney_reviewed`, `shred_cause`	How the session ended
`signer`	Secure Enclave or software

"Clean" has a precise meaning

A receipt is clean only when no internet socket was observed, the vault was verified zero, and the OS confirmed that the session process exited. A session that crashed before reporting a shred receives a receipt that is never clean.

Signature

ECDSA P-256 over deterministic bytes: sorted keys, no whitespace, ISO-8601 UTC seconds. Formatting or key order cannot change a verdict. The key ID is the first 16 hex characters of the SHA-256 of the public key.

VERIFIED valid, duplicate, tampered, software-key, unknown-key and wrong-Mac uploads each behaved as specified against the real control plane; the signature was also checked independently in Python.

What a receipt does not prove

It does not report bytes: it records whether an internet socket ever existed. `memset_zero_verified` means the vault was verified zero, not that no copy exists anywhere. `human_attorney_reviewed` records an approval, not the quality of the review. The signing key is not yet bound to the app's code identity, so a receipt proves that this Mac's key signed it, not that an unmodified Vaulta build did.

SECTION 8

8 Supply chain and models

Permissive licenses only, US or EU-origin model weights, and every external artifact pinned to an exact version or commit.

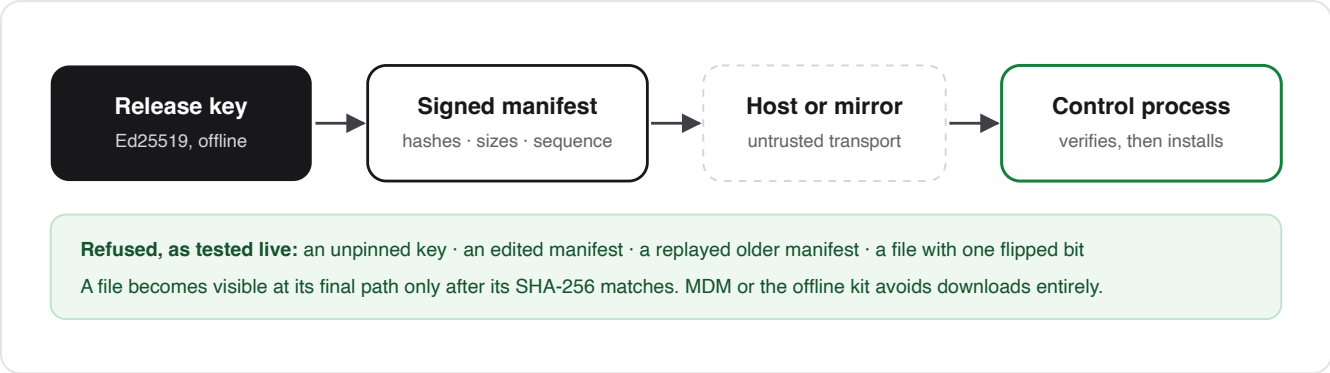


Figure 6. Trust comes from the pinned signature, not the transport. A compromised mirror can delay an update but cannot forge one.

Models

MODEL	ROLE	PUBLISHER	LICENSE	SIZE	PINNED TO
Whisper base.en	Speech to text (bundled)	OpenAI; Core ML by Argmax	MIT	147 MB	`0f63a780`
Whisper large-v3-turbo	Speech to text (default)	OpenAI; Core ML by Argmax	MIT	627 MB	weight-file SHA-256
Silero VAD v6.0.0	Voice activity	Silero; Core ML by FluidInference	MIT	0.9 MB	`b419383c`
Phi-3.5-mini-instruct, 4-bit	Memo drafting	Microsoft; MLX by mlx-community	MIT	2.15 GB	`7b2052fd`

Status

ITEM	DETAIL	STATUS
Signed manifest	Ed25519 with pinned keys, SHA-256 before install, persisted sequence against replay. Tested live with stand-in files over plain HTTP.	VERIFIED
Production signing key	None issued yet, so none is built into the client	OPEN
Third-party conversions	Silero Core ML and Phi MLX builds come from third-party repositories; we intend to reproduce both conversions before release	OPEN
Pinned hashes	Installed speech weights, tokenizer and Phi-3.5 matched our records. Two tree hashes did not reproduce (finding F-05).	OPEN
Receipt hash coverage	Receipt model hashes cover one large weight file per model; a per-file manifest is still to do	OPEN

SECTION 9

9 Control plane

The control plane holds organization policy, device public keys, signed receipts and the admin audit log. It never receives audio or transcripts.

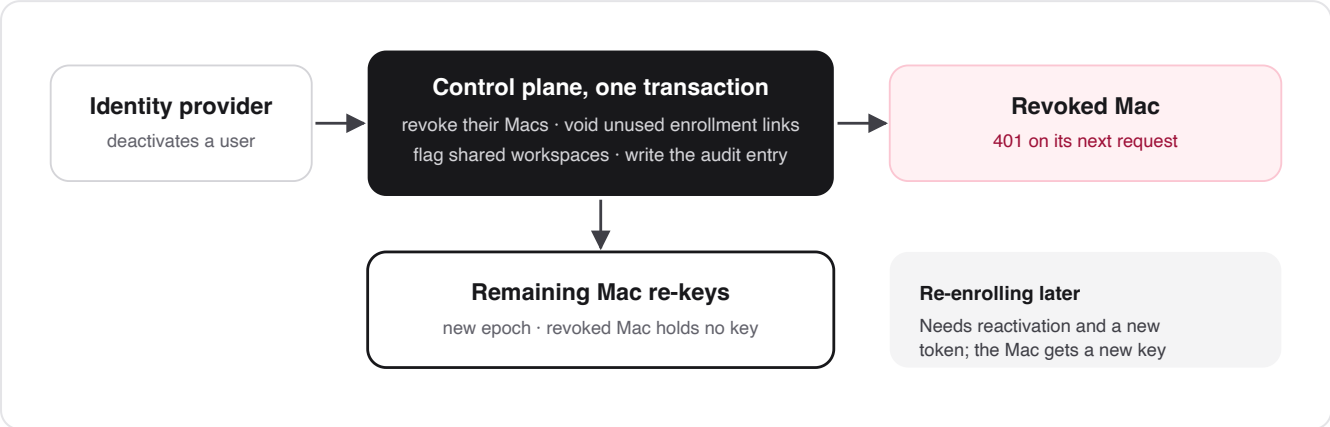


Figure 7. Deprovisioning and revocation. Tested live with the real Mac app, a scripted second device and a simulated identity provider.

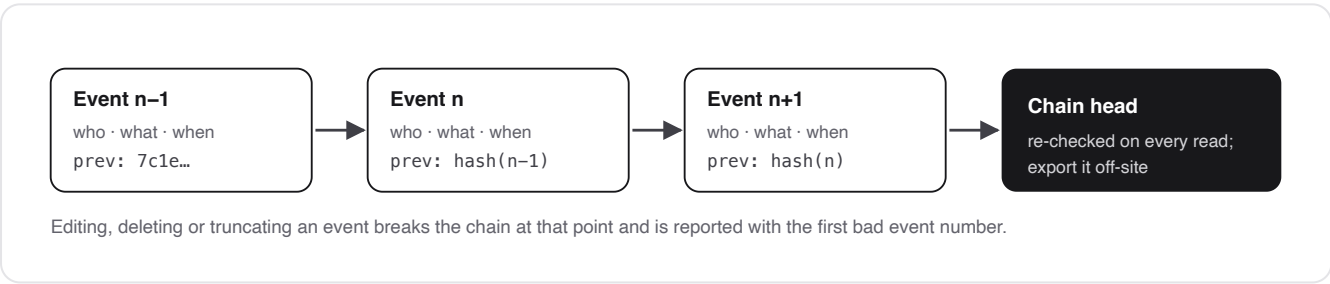


Figure 8. The admin audit log is hash-chained, so it shows tampering. It does not prevent tampering by someone with full database write access.

CONTROL	DETAIL	STATUS
Leavers lose access	SCIM deactivation or deletion revokes every Mac enrolled under the person's email and voids their unused enrollment links	VERIFIED
Revocation reaches workspaces	A revoked Mac is refused and removed from every Deal Room workspace, which is flagged for re-keying	VERIFIED
Every admin action is on record	Admin and SCIM changes, failed logins and exports are logged in the same transaction as the change, under a per-person identity	VERIFIED
The record shows tampering	Edited, deleted and truncated entries were each reported with where the chain broke	VERIFIED
Audit exports you can check	JSON with a content digest, spreadsheet-safe CSV, or PDF with an offline-checkable Ed25519 signature and an optional PAdES signature	VERIFIED
Production hardening	Migrations and the suite pass on PostgreSQL 16; failed-login limits hold across servers; HTTPS can be required with HSTS; database TLS fails closed	VERIFIED

SECTION 10

10 Device identity and updates

The product ships few updates, and every one is visible, explained and verified. Today there is no updater in the app; administrators install builds through MDM or the offline kit.

What we tested ahead of an updater

QUESTION	RESULT	STATUS
Does the device key survive an app update?	Yes. A Secure Enclave key created by one build was loaded and used by a different ad-hoc build, by a build with the same self-signed certificate, and by a build with a different self-signed certificate.	VERIFIED
Does the microphone permission survive?	Not with ad-hoc signing: macOS asks again after every rebuild. With a stable self-signed certificate the permission was kept.	OPEN
What does Gatekeeper do with a downloaded build?	It assessed a quarantined self-signed copy as rejected, exactly as it did an ad-hoc copy. A self-signed certificate does not change first launch.	OPEN
What happens if a stored key is unusable?	The next meeting does not start and the key file is left untouched; Vaulta never silently creates a replacement (finding F-02).	VERIFIED

Proposed update design

NO SILENT UPDATES

The user or the organization's administrator sees what changes and approves it. Never during a meeting.

A SMALL NETWORK PATH

One fixed host, one static signed file, no identifiers sent. The session process still has no network at all.

TRUST FROM SIGNATURES

A signed release manifest with a rollback guard, two pinned release keys, and an emergency revocation list.

NOTIFY-ONLY FIRST

Version one verifies and notifies; installation stays with MDM or the user. In-app install comes only after a stable signing identity is in place.

Decisions still open

Whether notify-only is enough for now, whether to adopt a stable self-signed signing certificate, who holds the offline release key, and how long an administrator may hold an update. These are recorded in architecture decision record 0002.

SECTION 11

11 Findings log

Issues surfaced by the live testing on 14 November 2026. Severity uses the scale in section 1.

F–01 A refused vault write could hand its nonce to the next record

MEDIUM FIXED

DESCRIPTION	The vault's sequence counter, which forms the AES-GCM nonce, advanced only after a record was stored. When a sealed record was refused because the segment was full, the next record reused its nonce under the same key.
IMPACT	The refused ciphertext was discarded and never stored, so no two stored records shared a nonce. Exploiting it would require the discarded ciphertext, which existed only briefly in ordinary process memory beside its own plaintext.
RESOLUTION	The counter is now used up before sealing, so a refused write leaves a gap in the numbering instead of passing its nonce on.
VERIFICATION	Three regression tests, each failing on the old code. Live re-runs: 1,131,072 distinct nonces; over a million concurrent attempts with most refused produced no repeat in debug, release and ThreadSanitizer builds.

F–02 An unusable device or memo key was silently replaced

MEDIUM FIXED

DESCRIPTION	If a stored signing key or memo-wrapping key could not be loaded, the client treated it as a first install and created a new one.
IMPACT	The Mac's identity would change without notice, and a new memo-wrapping key would make every stored memo permanently unreadable. Discovering the problem after a meeting would also lose the approved memo.
RESOLUTION	A key file that is present but unusable is now refused and left untouched, both keys are checked before a meeting starts, and the person sees a plain message to contact their administrator.
VERIFICATION	Unit tests, plus a live run of the built app: corrupt signing and memo keys each stopped the meeting before any session process started, with the files byte-identical afterwards.

F–03 A revoked Mac could never be enrolled again in the same organization

LOW FIXED

DESCRIPTION	The database allowed one registration per Mac per organization for good, including revoked ones, so re-enrolling a revoked Mac was refused as "already enrolled".
IMPACT	Availability and administration: a returning employee's laptop, or a Mac recovering from an unusable key, could not rejoin.
RESOLUTION	Migration 0006 enforces one *active* registration per Mac per organization. Revoked rows are kept for the audit trail, and a revoked key can still never be registered again.
VERIFICATION	Four API tests and a database-level test, each failing without the fix. Live with the real app: after reactivation the same Mac enrolled again and its old key stayed refused. Migration verified on PostgreSQL 16, including its downgrade guard.

F-04 Forwarded headers from the local host were trusted under the web server's defaults

LOW · MITIGATED

DESCRIPTION	With HTTPS required and no trusted proxy configured, a plain-HTTP request from the same machine carrying a forged `X-Forwarded-Proto: https` was accepted. The cause was uvicorn's default of trusting forwarded headers from 127.0.0.1, not the application's logic.
IMPACT	Requires a process on the server itself.
RESOLUTION	The deployment guide now requires `--no-proxy-headers`, so only the application decides which proxies to believe.
VERIFICATION	With the flag, the same forged request was refused with 403.

F-05 Two recorded model hash pins could not be reproduced

LOW · OPEN

DESCRIPTION	The recorded tree hashes for Silero VAD and Whisper base.en did not match any layout we could compute from the installed copies. The other pinned hashes matched exactly.
IMPACT	Record-keeping: the software bill of materials could not be used to confirm those two models.
RESOLUTION	The bill of materials now records the observed hashes and says the original pins were not reproducible. Comparing against a fresh download of the upstream commits is still to do.

F-06 The public brief overstated protection against debugger attach

INFO · OPEN

DESCRIPTION	The security brief says deny-attach is for release builds. No build blocks debugger attach today, and no build uses hardened runtime.
IMPACT	Documentation accuracy only. Memory protection against other local processes is already listed as out of scope.
RESOLUTION	Correct the wording to say debugger attach is not blocked in any build yet, and that hardened runtime, library validation and deny-attach arrive with a Developer ID.

SECTION 12

12 Known limits and open items

We would rather you read these here than discover them in a pilot.

Limits of the design

LIMIT	DETAIL
The control process is not sandboxed	It never sees audio or verbatim text, but holds the network connection and the memo store.
Other local processes reading our memory is out of scope	Defending against it needs hardened runtime and library validation, which need a Developer ID. Debugger attach is not blocked in any build yet.
The receipt key is not bound to the app	A receipt proves that this Mac's Secure Enclave key signed it, not that an unmodified build did. Binding needs a stable signing identity.
Approval does not prove careful review	The click is recorded in the receipt. It is a product control, not a technical one.
Revocation cannot reach into the Mac	Memos already stored on a revoked Mac stay readable there until its keys are deleted.
The audit chain shows tampering; it does not prevent it	Someone with full database write access could rebuild the chain. Export the log regularly and keep its latest hash elsewhere.
Only failed credentials are rate-limited	Successful traffic is not throttled; set a request limit at your proxy too.

Not done or not yet verified

ITEM	DETAIL
Signing and notarization	Builds are ad-hoc signed; Macs that run only Developer ID software refuse them.
Single sign-on from the Mac app	The control plane accepts Okta and Entra sign-in; the Mac enrolls with a one-time link.
Live meeting capture	One live test of Zoom, Teams and Meet; remote participants appear as one speaker.
Screen-share cloak	The Review window opts out of screen sharing; untested against Zoom or QuickTime.
Key hand-off on a Developer ID build	Tested on one Mac with ad-hoc and self-signed builds only.
Microphone permission after an update	Resets with ad-hoc signing; MDM pre-approval is not yet tested.
Updates	No updater in the app yet; a notify-only design is proposed.
Identity providers, live	Tested with Okta- and Entra-style requests, not a live tenant; users only, not groups.
PDF signer trust	The PAdES signature is validated with pyHanko, not yet opened in Acrobat.
Continuous PostgreSQL testing	Passes on PostgreSQL 16 locally; a CI job on every change is still to do.
Model supply chain	Production signing key not yet issued; third-party conversions to be reproduced from upstream; two hash pins to re-verify (F-05); per-file hashes in receipts still to do (section 8).
Certifications and third-party assessment	None are claimed in this report.

APPENDIX A

A Evidence log

Each live check from 14 November 2026: what ran, the control that proves the check could fail, the result, and its limits.

CHECK	METHOD AND CONTROL	RESULT	LIMITS
Memory placement	Whole-process scan with the kernel's per-region pinned flag while the real vault sealed 120 records and ran 300 decrypts. Control: plaintext placed in a pinned page on purpose was reported.	0 plaintext hits in pinned regions (debug and release). Key: exactly 1 hit, in a pinned page; 0 after shred.	Test program; key search in debug only
Ciphertext binding	47 forged handles, 130 single-byte changes, and an independent Python AES-GCM check. Control: untouched and rebuilt handles opened.	0 accepted; 43 refused by authentication, 4 by range check; 130 of 130 byte changes refused; 26 altered-data cases and 20 swaps refused.	Debug build, test program
Nonce uniqueness	One million transcript and 166,667 audio records, then refused writes, then 12 concurrent writers with a filling segment.	1,131,072 distinct nonces. After fix F-01: a refused number is never stored; up to 1,086,758 concurrent attempts with 0 repeats.	Test program
Concurrent writers	12 writers, 4 readers, 3 s; 450 shred-during-write trials; ThreadSanitizer. Control: an unlocked counter gave over a million duplicates and was flagged as a race.	0 duplicates, 0 corrupted records, 0 late writes, 0 races reported.	Time-based sampling
Zero egress	Five consecutive release-build sessions in the production sandbox with a `nettop` sampler. Control: a renamed process moving 8 MB was recorded.	0 internet sockets, shred verified zero and no network flows in every run.	Demo audio, one Mac
Socket watchdog	Real watchdog in a sandboxed binary; unsandboxed baseline.	Local IPC ignored; TCP socket flagged despite `EPERM`; documented blind spot reproduced.	Test program
Build boundary	Planted a network call in session-side code.	The build test failed and named the file; passed after removal.	Own code only
Model manifest	Real updater against a local HTTP server, six scenarios.	Valid install matched its hash; flipped bit, replay, unpinned key and edited manifest refused.	Stand-in files, plain HTTP
Unusable keys	Built app with a corrupt signing key, then a corrupt memo key.	Both refused before any session started; files unchanged.	One Mac
Key hand-off	Ad-hoc builds, same and different self-signed certificates.	Every build loaded and used the original key.	No Developer ID
Microphone permission	Two builds, ad-hoc, then two with one self-signed certificate.	Reset with ad-hoc; kept with the stable certificate.	One Mac

CHECK	METHOD AND CONTROL	RESULT	LIMITS
Revocation	Real app plus a scripted second device on a real server.	Revoked device got 401 everywhere; the real app re-keyed the workspace.	Scripted second device
SCIM deprovisioning	Okta- and Entra-style requests from a simulated identity provider.	Mac revoked, unused link voided, audit entries written; another user unaffected.	Not a live tenant
Audit chain	Edited, deleted and truncated events on a running server. Control: an untouched copy.	Each reported with the first bad event; untouched copy verified.	Head rewrite not caught
Audit exports	PDF, JSON and CSV over HTTP; offline verification; pyHanko.	Valid file verified; one changed figure failed; PAdES intact over the whole file.	Not opened in Acrobat
PostgreSQL 16	Throwaway cluster, two servers, HTTPS and TLS settings.	Migrations up, down and up; shared lockout; HSTS; TLS failed closed.	TLS success not shown

APPENDIX B

B Verifying our claims yourself

You do not need to take this report on trust. Each of these can be checked with your own tools.

A session receipt

1. Take the `receipt` object and serialize it with sorted keys, no whitespace and UTF-8, keeping timestamps as given.
2. Verify the `signature` as ECDSA P-256 with SHA-256 over those bytes, using the device's public key.
3. Confirm the key ID equals the first 16 hex characters of the SHA-256 of that public key.
4. Treat the session as clean only if the egress verdict is `none_observed`, sockets are 0, and both `memset_zero_verified` and `session_process_exited` are true.

A signed audit PDF

Fetch the public key from `GET /v1/compliance/audit-key`, then run:

```
python -m app.auditpdf verify report.pdf <public-key-base64>
```

The checker re-reads the text from the PDF itself, so a changed figure anywhere in the report fails. When a document-signing certificate is configured, PDF readers also show a PAdES signature that breaks on any change to the file.

The admin audit log

Every read of the log re-checks the whole hash chain. Export the log on a schedule and store its latest head hash somewhere the database administrators cannot reach; a later export whose chain does not extend that head has been rewritten.

Network behaviour on your own Mac

During a session, watch the session process with the macOS `nettop` tool or your endpoint agent. Our runs saw no network flows for it, and the sandbox refuses any connection it attempts.

APPENDIX C

C Glossary

AES-256-GCM	Authenticated encryption: it keeps data secret and detects any change to it or to its associated data.
Authenticated data	Information bound into the encryption without being encrypted. Here: segment, tag, timestamp and sequence.
App Sandbox	The macOS mechanism that limits what a process can do. The session process has no network entitlement.
Control plane	Vaulta's server: enrollment, policy, receipts and the admin audit log. It never receives audio or transcripts.
Deal Room workspace	A shared glossary of code names, end-to-end encrypted to the member Macs' keys.
`mlock`	Pins memory so the operating system never pages it to disk.
Nonce	A number used once per encryption under a given key. Reusing one weakens AES-GCM.
PAdES	The standard for signatures embedded in PDF files, shown by PDF readers.
Receipt	A signed record of how a session ended, with no audio or transcript content.
SCIM	The standard identity providers use to tell applications when people join, change or leave.
Secure Enclave	The Mac's hardware key store. Keys created there cannot be exported.
Shred	Destroying the vault key, zeroing the vault, and reading it back to confirm.
ThreadSanitizer	A compiler tool that detects data races between threads at run time.
Vault	The encrypted, pinned memory region that holds raw audio and the verbatim transcript.

This report describes the Vaulta macOS client and control plane as built on 14 November 2026. It is a first-party assessment, not a warranty, a certification, or a substitute for your organization's own security and legal review. Mac, Apple silicon and Secure Enclave are trademarks of Apple Inc.